

SIO 2^{ème} année option SLAM
BLOC2 : Réalisation et maintenance de composants logiciels
GSB : Application avec appel de sous-activités sous Android

1. Maquette du projet à réaliser	2
➤ Fenêtre principale.....	2
➤ Fenetre AjoutEchantillon	2
➤ Fenetre ListeEchantillons.....	3
➤ Fenetre MajEchantillon.....	3
2. Réalisation de la fenêtre principale.....	4
➤ Création du menu	4
➤ Gestion des évènements associés au menu	5
3. Gestion des sous-activités.....	6
➤ Réalisation de la fenetre ajout_echant	6
➤ Gestion de l'ouverture de la sous-activité ajout_echant	7
➤ Réalisation de la fenetre ListeEchant.....	9
➤ Réalisation de la fenetreMajEchant	9

1. Maquette du projet à réaliser

Pour gérer des articles d'une société, on dispose de la classe **Echantillon.java** qui sera décrite ultérieurement.

On désire pouvoir consulter et mettre à jour un stock d'échantillon de compléments alimentaire partir d'une application Android.

L'application va être composée de 3 fenêtres :

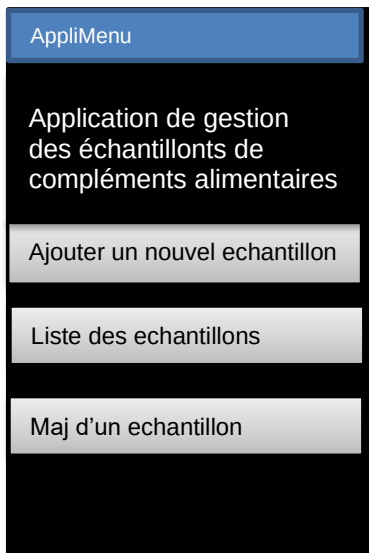
- La fenêtre principale qui permet d'appeler les 3 autres fenêtres
- Une fenêtre dédiée à la saisie d'un échantillon
- Une fenêtre dédiée à l'affichage des échantillons
- Une fenêtre dédiée à la mise à jour du stock d'échantillon

Les articles seront stockés dans un premier temps en mémoire dans une liste.

➤ Fenêtre principale **activity_main.xml**

Cette fenêtre permettra d'appeler les 3 autres fenêtres par 2 moyens :

- En cliquant que l'un ou l'autre des boutons
- En cliquant sur un des items du menu qui apparaîtra en cliquant à droite de l'écran sur la touche menu de l'émulateur .



Titre de l'application

menuTextViewTitre « Application de gestion des échantillons de compléments alimentaires »

menuButtonAjoutEchantillon « Ajouter un nouvel échantillon » : en cliquant sur le bouton :

- La sous fenêtre AjoutEchantillon s'ouvre

menuButtonListeEchantillons « liste des echantillons » : en cliquant sur le bouton :

- La sous fenêtre ListeEchantillons s'ouvre

menuButtonMajEchantillons « Maj d'un echantillon » : en cliquant sur le bouton :

- La sous fenêtre MajEchantillon s'ouvre

➤ Fenetre ajout_echantillon.xml

Fenêtre qui permet de saisir les renseignements concernant un echantillon.



Titre de l'application

ajoutTextViewEchantillon « Saisie d'un échantillon : »

ajoutEditTextCode « Saisir ici un code »

ajoutEditTextLib « Saisir ici un libellé »

ajoutEditTextStock « Saisir ici un stock »

ajoutButtonAjouter « Ajouter » : en cliquant sur le bouton :

- le message « code, libellé, quantité » s'affiche dans le TextViewMessage

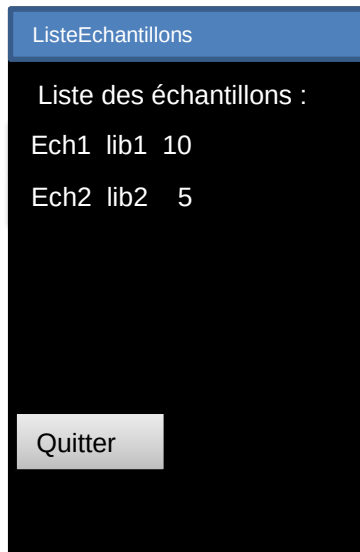
ajoutButtonQuitter « Quitter » : en cliquant sur le bouton :

- la fenêtre se ferme

ajoutTextViewMessage « message »

➤ Fenetre liste_echantillons.xml

Fenêtre qui permet de lister tous les échantillons.



Titre de l'application

listeTextViewTitre « Liste des échantillons : »

listeListViewEchantillon « contient les renseignements concernant le stock d'échantillons »
on est obligé d'utiliser une ListView car on ne connaît pas le nombre exact d'échantillons.

listeButtonQuitter « Quitter » : en cliquant sur le bouton :

- la fenêtre se ferme

➤ Fenetre maj_echantillon.xml

Fenêtre qui permet de mettre à jour le stock pour un échantillon.



Titre de l'application

majTextViewTitre « MAJ d'un échantillon : »

majEditTextCode « Saisir ici un code »

majEditTextQte « Saisir ici une quantité »

majButtonSupprimer « Supprimer » : en cliquant sur le bouton :

- Le stock concernant le code saisi est mis à jour
- le message « quantité supprimée » s'affiche dans le TextViewMessage

majButtonAjouter « Ajouter » : en cliquant sur le bouton :

- Le stock concernant le code saisi est mis à jour
- le message « quantité ajoutée » s'affiche dans le TextViewMessage

majButtonQuitterListe « Quitter » : en cliquant sur le bouton :

- la fenêtre se ferme

majTextViewMessage « message »

2. Réalisation de la fenêtre principale

➤ Création du menu

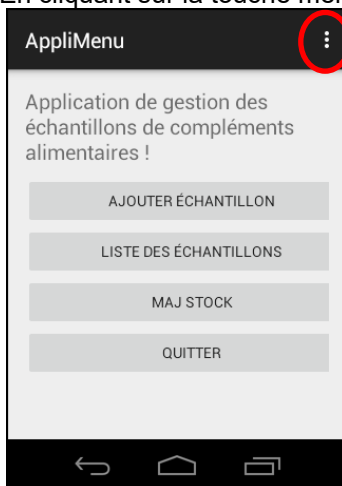
Pour construire un menu il suffit d'ajouter la méthode **onCreateOptionsMenu**. Selon les versions, cette méthode existe déjà dans le code (**voir cours correspondant**)

```
@override
public boolean onCreateOptionsMenu(Menu menu) {
    // Inflate the menu; this adds items to the action bar if it is present.
    getMenuInflater().inflate(R.menu.main, menu);
    return true;
}
```

Si le menu n'apparaît pas au départ, vérifiez que le thème android choisi est compatible avec les menus ou ajouter la ligne suivante dans le fichier **AndroidManifest.xml** :

```
android:theme="@style/Theme.AppCompat.Light.DarkActionBar"
```

En cliquant sur la touche menu de l'émulateur on peut constater qu'un menu existe déjà :



Ce fichier **menu** se trouve dans le fichier **main.xml** du dossier **res/menu**.

Pour créer un menu :

- ➔ Créez un nouveau fichier de type menu **main.xml** dans un dossier menu
- ➔ Rajouter dans le fichier XML les différents menus, sous-menu et item comme ci-dessous :

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item android:id="@+id/echantillons"
        android:title="Echantillons">

        <menu android:id="@+id/sousMenu">
            <item android:id="@+id/ajout"
                android:title="Ajout" />
            <item android:id="@+id/liste"
                android:title="Liste" />
            <item android:id="@+id/maj"
                android:title="Maj" />
        </menu>
    </item>

    <item android:id="@+id/quitte"
        android:title="Quitter"
        android:icon="@android:drawable/ic_menu_close_clear_cancel" />

</menu>
```

<menu ...> ...</menu>

→ permet de définir un menu ou un sous-menu

<item android:id="@+id/quitter"

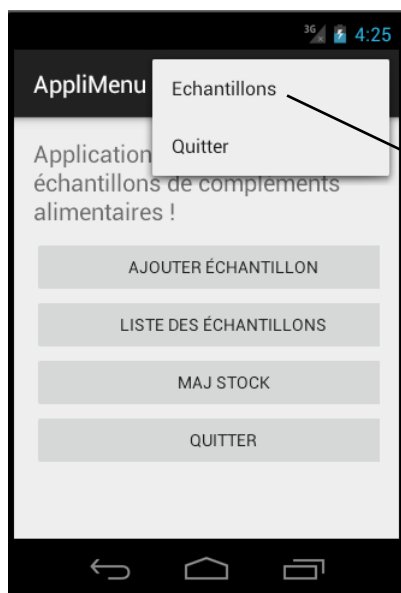
→ option d'un menu et son identifiant

android:title="@string/Quitter" → chaîne de caractère associée à l'option du menu

android:icon="@android:drawable/ic_menu_close_clear_cancel" />

→ icône associée au menu

le site <http://androiddrawableexplorer.appspot.com/> propose les icônes officiels d'android.



ouverture fenêtre Ajout !

Ajout
Liste
Mise à jour

en cliquant sur Echantillons on obtient un sous-menu →

➤ Gestion des événements associés au menu

La méthode `onOptionsItemSelected()` doit être complétée pour gérer les événements liés aux menus.
(voir cours correspondant)

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    int itemId = item.getItemId();
    //dans le cas où on clique sur l'item quitter
    if (itemId == R.id.quitter){
        finish();
        return true;
    }else if(itemId == R.id.ajout){
    //dans le cas où on clique sur l'item ajouter
        Toast.makeText(getApplicationContext(), "ouverture fenêtre Ajout !", Toast.LENGTH_LONG).show();
        return true;
    }else if(itemId == R.id.liste){
    //dans le cas où on clique sur l'item liste
        Toast.makeText(getApplicationContext(), "ouverture fenêtre Liste !", Toast.LENGTH_LONG).show();
        return true;
    }else if(itemId == R.id.maj){
    //dans le cas où on clique sur l'item maj
        Toast.makeText(getApplicationContext(), "ouverture fenêtre Maj !", Toast.LENGTH_LONG).show();
        return true;
    }
    return true;
}
```

Pour l'instant en cliquant sur l'option Ajout ou liste, une popup apparait avec un message. Il faudra d'abord créer les 3 sous-fenêtres avant de gérer leur ouverture.

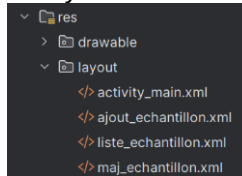
3. Gestion des sous-activités

➤ Réalisation de la fenetre ajout_echantillon.xml

Dans un premier temps créer l'interface correspondant à la fenêtré ajout_echant :



➔ Créer un nouveau fichier **ajout_echantillon.xml** dans le dossier res\layout



➔ Construire l'interface

➔ Associer à chaque zone de saisie, de texte et bouton un id et une variable

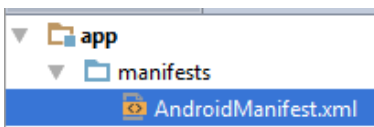
➔ Créer un nouveau fichier **AjoutEchantillon.java** dans le package associé à l'application

```
package com.example.applimenu;
```

```
import android.app.Activity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
```

```
public class AjoutEchantillon extends Activity {
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.ajout_echantillon);
        Button buttonQuitter = (Button)findViewById(R.id.ajoutButtonQuitter);
        buttonQuitter.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                // TODO Auto-generated method stub
                finish(); //fermeture de la fenêtré
            }
        });
    }
}
```

Il faut déclarer cette sous-activité dans le manifeste **AndroidManifest.xml** (à la racine) afin qu'elle puisse s'exécuter. Pour cela il suffit de dupliquer la balise <activity...> déjà existante.



```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.applimenu" >

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name="AjoutEchantillon"
            android:label="@string/app_name" >
        </activity>
    </application>

</manifest>

```

Ajout de la nouvelle Activité

➤ Gestion de l'ouverture de la sous-activité ajout_echantillon

Modifier le code de la méthode **onOptionsItemSelected** de la classe **MainActivity.java**.
Pour démarrer une activité, il faut instancier un Intent.

Un Intent est un message asynchrone sous Android permettant d'activer une activité (entre autre). Le message peut être porteur d'informations.

```

@Override
    public boolean onOptionsItemSelected(MenuItem item) {
        int itemId = item.getItemId();
        //dans le cas où on clique sur l'item quitter
        if (itemId == R.id.quitter){
            finish();
            return true;
        }else if(itemId == R.id.ajout){
            //dans le cas où on clique sur l'item ajouter
            Toast.makeText(getApplicationContext(), "ouverture fenêtre Ajout
            !", Toast.LENGTH_LONG).show();
            Intent intentAjout = new Intent(MainActivity.this, AjoutEchantillon.class);
            startActivity(intentAjout);

            return true;
        }
    }

```

Modifier le code de la méthode **onCreate** pour gérer l'évènement click sur le bouton **ajoutButtonAjouter**.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);
    Button buttonAjoutEchant =
(Button) findViewById(R.id.ajoutButtonAjouter);

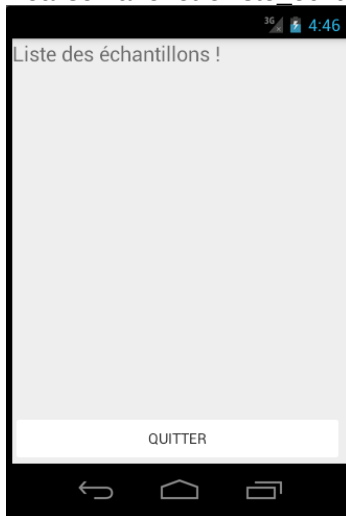
    buttonAjoutEchant.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            Intent intentAjout = new Intent(MainActivity.this,
AjoutEchantillon.class);
            startActivity(intentAjout);
        }
    });
}
```

Maintenant en cliquant soit sur le menu **Echantillons → Ajouter** ou sur le bouton **ajouter un nouvel echantillon** on obtient l'ouverture de la fenêtre **ajout_echantillon**.

→ Complétez désormais le reste des fonctionnalités à développer dans cette fenêtre

➤ Réalisation de la fenetre liste_echantillon.xml

Réaliser la fenêtre liste_echant afin qu'elle ait l'aspect suivant :

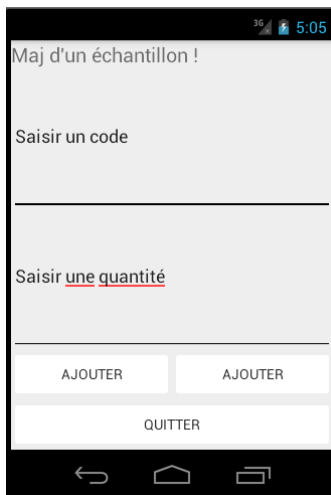


Pour l'instant seuls la zone de texte avec le titre et le bouton quitter apparaissent car aucune données n'est encore disponible.

Gérer les événements afin que cette fenêtre s'ouvre lorsqu'on clique sur le bouton **Liste des échantillons** ou le menu **liste** dans la fenêtre principale.

➤ Réalisation de la fenetre maj_echantillon.xml

Réaliser la fenêtre maj_echant afin qu'elle ait l'aspect suivant :



Gérer les événements afin que cette fenêtre s'ouvre lorsqu'on clique sur le bouton **Maj stock** ou le menu **Maj** dans la fenêtre principale.